Bases de Datos

Clase 15: Data Science, Pandas y Data Engineering



Hasta ahora

Recapitulación del curso

A lo largo del curso se ha estudiado cómo:

- Diseñar un modelo de datos
- Transformarlo a tablas en formas normales
- Almacenar los datos en una BD
- Formular consultas
- Calcular su complejidad
- Proteger su privacidad
- Usar herramientas externas de apoyo

¿Y ahora? **Data Science**

Parte I

Data Science



Data Science

Definición

> *La ciencia de datos es un enfoque multidisciplinario para extraer conocimientos prácticos de los grandes y crecientes volúmenes de datos recopilados y generados por las organizaciones. Abarca la preparación de los datos para su análisis y procesamiento, la ejecución de análisis avanzados y la presentación de resultados que revelan patrones y permiten a las partes interesadas extraer conclusiones informadas.*

Fuente: **IBM**

Data Science

Áreas que abarca



Data Analysis



Machine Learning



Deep Learning



Inteligencia Artificial (AI)



Computer Vision

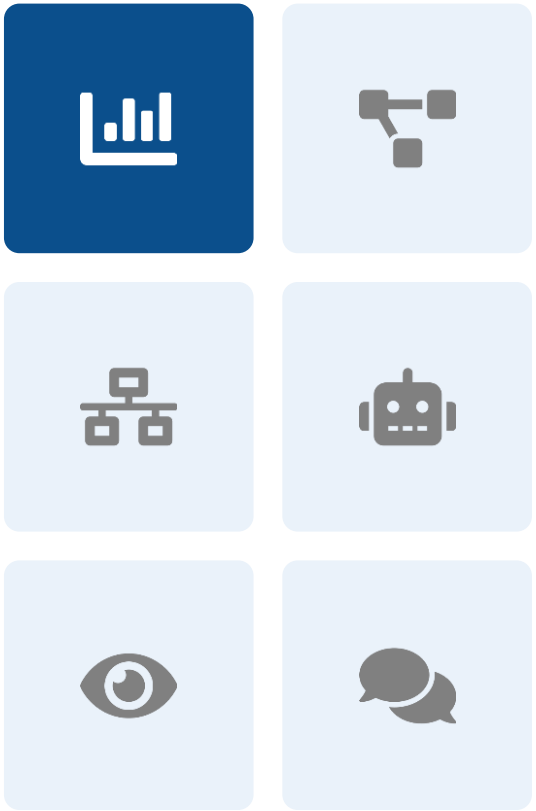


Natural Language Processing

…y varias otras áreas.

Data Science

Foco de esta clase



Data Analysis es el área de Data Science que se abordará en esta clase.

Se estudiará **Pandas**, la biblioteca de Python más utilizada para el análisis de datos.

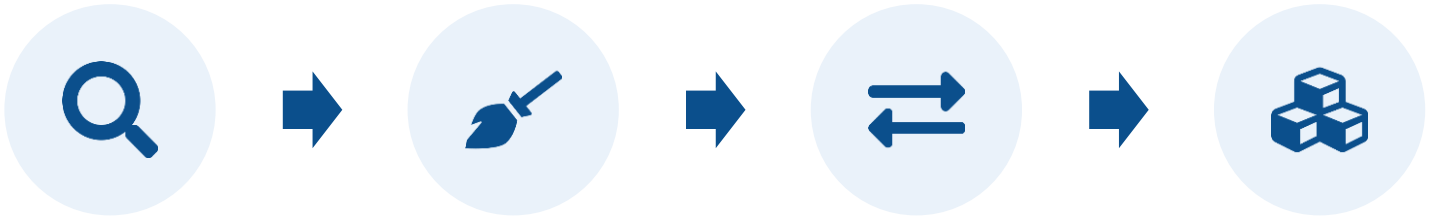
Las demás áreas se abordan, en general, en otros cursos.

Data Analysis

Data Analysis

Definición

Proceso de **inspección, limpieza, transformación y modelamiento** de datos, cuyo objetivo es extraer información relevante para un fin determinado.



Resultado: **información relevante**

El proceso de Data Science

Etapas y roles



Collection | Cleaning | Exploratory Data Analysis | Model Building | Model Deployment

Roles

- Data Engineers
- Data Analysts
- Machine Learning Engineers
- Data Scientists

En general, se ve así

De la base de datos al reporte

1



Los datos se encuentran almacenados en una base de datos.

2



Se cargan en un entorno de trabajo (Jupyter Notebook, R, Tableau, etc.).

3



Se extrae información relevante para elaborar reportes, modelos, etc.

La biblioteca Pandas

Descripción

Pandas es una herramienta de código abierto para la manipulación y el análisis de datos: rápida, potente, flexible y fácil de usar, construida sobre el lenguaje de programación Python.



La biblioteca Pandas

Formatos de archivo soportados



`.xlsx, .xls, …`

Planillas de cálculo



`.csv`

Valores separados por comas



`.tsv`

Valores separados por tabulación



`.json`

JavaScript Object Notation

…y otros más. Lista completa en la **documentación de Pandas (I/O)**.

Pandas: DataFrames

La clase DataFrame

```
class pandas.DataFrame(
    data=None, index=None,
    columns=None, dtype=None,
    copy=None)
```

- Estructura tabular bidimensional, de tamaño mutable y potencialmente heterogénea.
- Posee ejes etiquetados (filas y columnas); las operaciones aritméticas se alinean según ambas etiquetas.
- Puede entenderse como un contenedor tipo diccionario de objetos **Series**. Es la estructura de datos principal de Pandas.

Pandas: DataFrames

Anatomía de un DataFrame

Índice

Nombres de columnas

	name	pop	pib
0	Metropolitana	7112808	24850
1	Valparaíso	1815902	14510
2	Biobío	1538194	13281
3	Maule	1044950	12695
4	Araucanía	957224	11064
5	O'Higgins	914555	14840

Tuplas (filas)

Ejemplo: población (pop) y PIB per cápita (pib) por región.

Pandas: DataFrames

Joins (método merge)

```
DataFrame.merge(right, how='inner',
    on=None, left_on=None, right_on=None,
    left_index=False, right_index=False,
    sort=False, suffixes=('_x', '_y'),
    copy=True, indicator=False,
    validate=None)
```



Combina objetos DataFrame o Series con nombre mediante un **join al estilo de una base de datos**.

Pandas: DataFrames

Joins: comportamiento de merge

Series con nombre

Un objeto Series con nombre se trata como un DataFrame de una sola columna con ese nombre.

Columnas con columnas

Si el join se realiza entre columnas, los índices de los DataFrames se ignoran.

Uso de índices

Si se unen índices con índices, o índices con columnas, el índice se conserva en el resultado.

Cross merge

En un cross merge no se permite especificar columnas de unión.

Pandas: DataFrames

Query

```
DataFrame.query(expr,
    inplace=False, **kwargs)
```

Consulta las columnas de un DataFrame mediante una **expresión booleana**.

Parámetro `expr : str`

Cadena con la consulta a evaluar. Es posible referirse a variables del entorno anteponiendo el carácter `@`, por ejemplo `@a + b`.

Pandas: DataFrames

Query: nombres de columnas

Las columnas cuyo nombre no es un identificador válido de Python (con espacios, signos de puntuación distintos de "_", o que comienzan con dígitos) deben encerrarse entre backticks.

Caso	Expresión en query
Variable del entorno a, sumada a la columna b	`@a + b`
Columna llamada "Area (cm^2)"	`` `Area (cm^2)` ``
Columna "a a", sumada a la columna b	`` `a a` + b ``

⚠ No es posible usar como nombres de columna palabras reservadas de Python (list, for, import, etc.).

Pandas: DataFrames

Group By

```
DataFrame.groupby(by=None, axis=0,
    level=None, as_index=True,
    sort=True, group_keys=True,
    squeeze=NoDefault.no_default,
    observed=False, dropna=True)
```



Agrupa un DataFrame mediante un **mapeo** o una **serie de columnas**.

Pandas: DataFrames

Group By: split-apply-combine

Una operación groupby combina tres pasos. Permite agrupar grandes volúmenes de datos y calcular operaciones sobre cada grupo.



Parte II

Data Engineering



Data Engineering

Pregunta inicial



¿Son los conceptos vistos hasta ahora todos los que se requieren para construir un sistema que gestione adecuadamente los datos?

Data Engineering

Motivación

En algunos casos, sí. Sin embargo, con el tiempo:



Aumenta la complejidad de los sistemas



En algunos casos, aumenta también la carga

Esto impone nuevos requisitos sobre las bases de datos, que suelen requerir esfuerzos de ingeniería dedicados para su correcto funcionamiento.

Data Engineering

¿Qué es?



Definición

Rama de la ingeniería de software dedicada a desarrollar y mantener software para transmitir, procesar, almacenar y acceder a los datos.



Rol de los data engineers

Proveer al resto de los miembros de la organización plataformas para acceder a los datos de forma confiable y eficiente.

Data Engineering

¿Por qué? Crecimiento y consumo



Cuando los sistemas (y las organizaciones) crecen, surge la necesidad de protocolos e infraestructura que mantengan los datos consistentes y accesibles 24/7, en distintos formatos.



Distintos stakeholders (***data scientists, business intelligence, developers***, etc.) consumen los datos de formas diferentes, y no es viable que todos consulten la base de datos SQL principal.

Data Engineering

¿Por qué? Procesamiento de datos

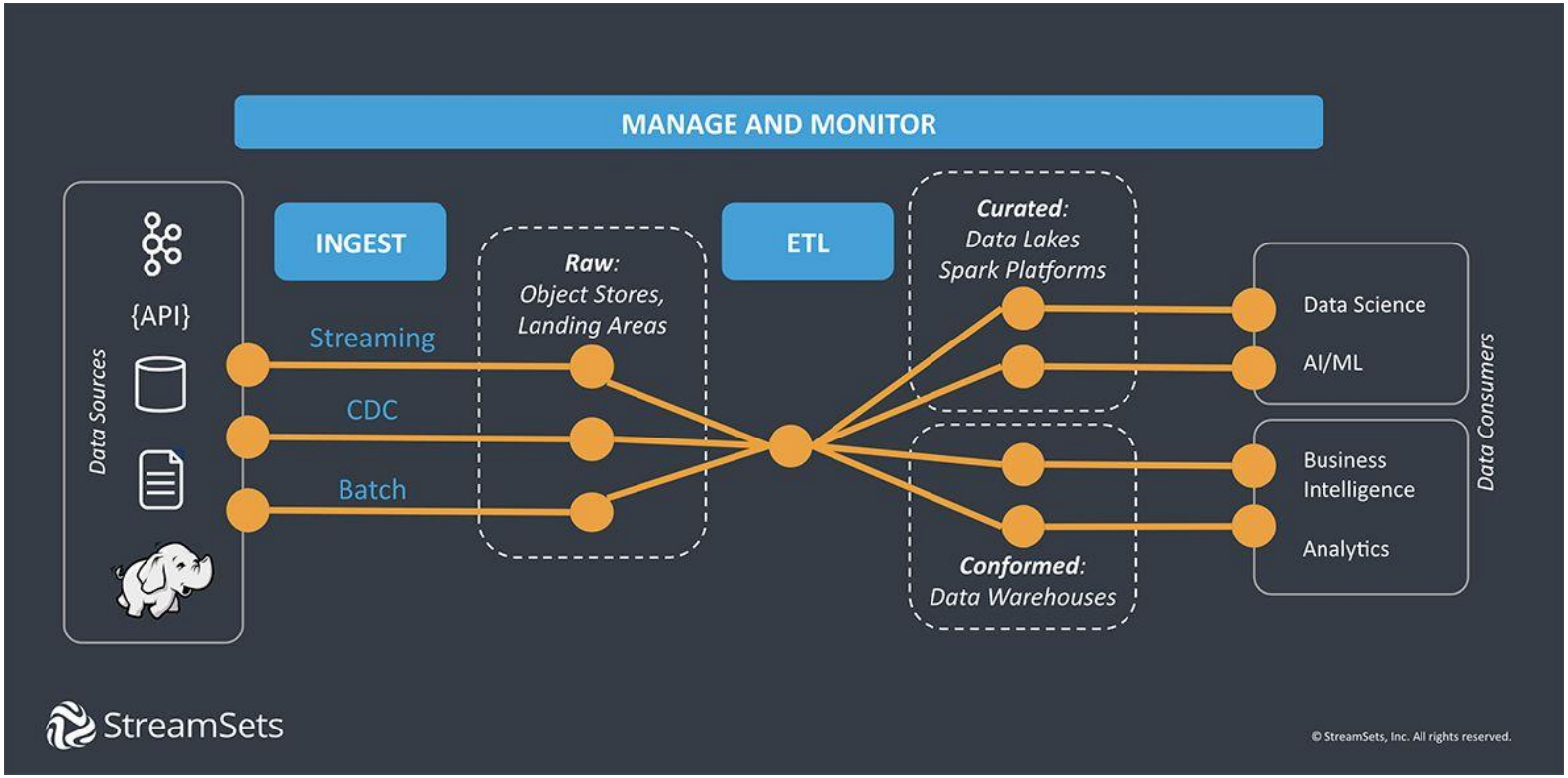
Todos los datos generados por un sistema deben ser extraídos, procesados y almacenados según lo requieran los objetivos de la organización.



Extraer **Procesar** **Almacenar**

Data Engineering

Ejemplo de un sistema



Fuentes de datos → ingesta → almacenamiento crudo → ETL → plataformas curadas y warehouses → consumidores.

Data Lakes y Warehouses

Dos patrones de consulta

Aplicación

Ejecuta muchas consultas de escritura y lectura, pero de baja complejidad: pocos joins y agregaciones.

Análisis

Los miembros de la organización que analizan el sistema ejecutan consultas con abundantes agregaciones y joins.

Data Lakes y Warehouses

Consultas analíticas

Ejemplo de consulta analítica:

> *¿Cuánto ha vendido en promedio cada tienda, cada mes, durante los últimos 12 meses?*

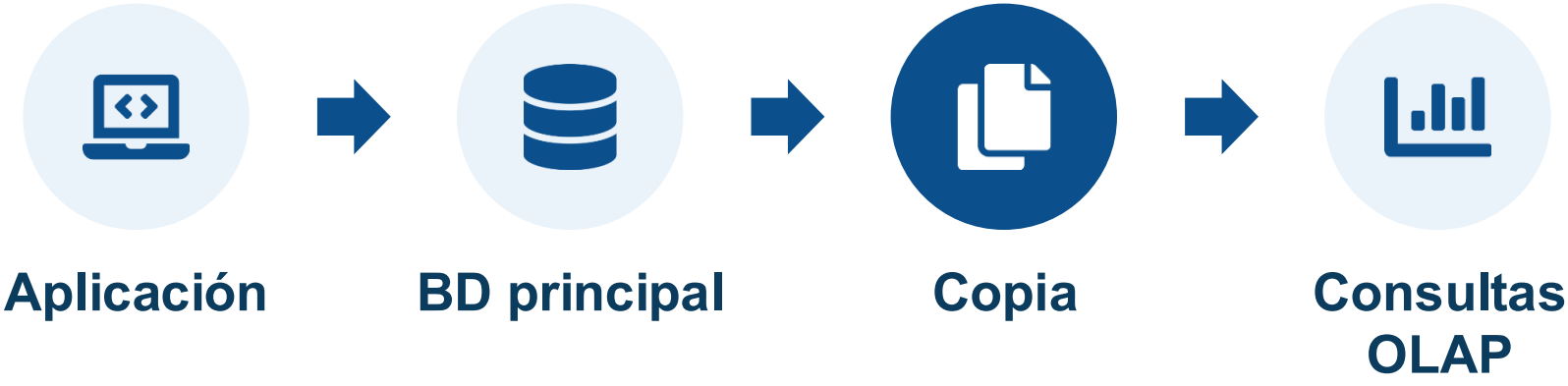


Estas consultas son **muy costosas para el DBMS** y pueden afectar la disponibilidad de la aplicación para los usuarios reales.

Data Lakes y Warehouses

OLAP

Este tipo de consultas se denomina **On-Line Analytical Processing (OLAP)**, y se busca ejecutarlas en otro lugar. Para ello se generan copias de la(s) base(s) de datos del sistema.



Data Lakes y Warehouses

Tipos de copia



Data lake

Copia de los datos tal cual se encuentran en el sistema.



Data warehouse

Copia preprocesada para un uso particular.

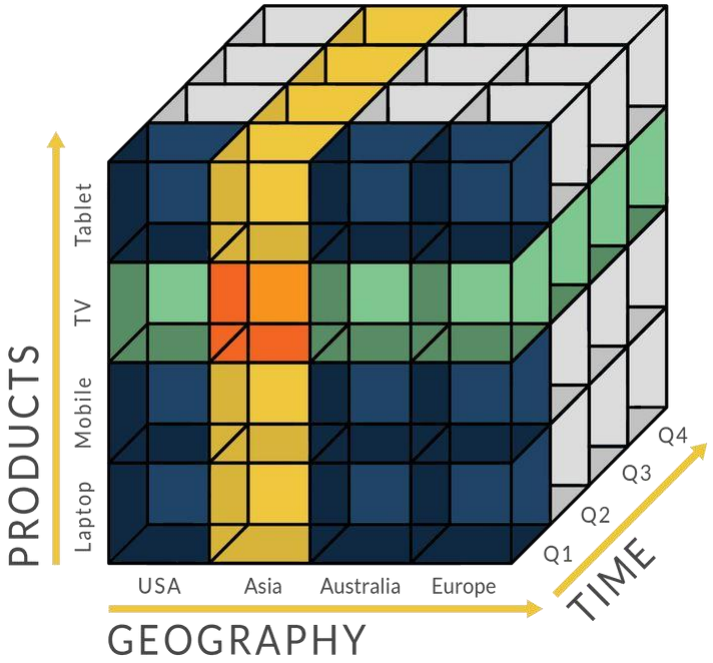


Estas copias no suelen estar actualizadas en tiempo real, pero pueden consultarse libremente y suelen conservar toda la información histórica del sistema (incluso aquella deprecada en la BD original).

Data Warehouses

OLAP Cube

- Un warehouse suele almacenar los datos preprocesados de manera que se faciliten las consultas que se ejecutarán sobre él.
- Una estrategia clásica es guardar los datos de forma **multidimensional**.



Data Warehouses

Bases de datos columnares

- Almacenan los datos en disco columna por columna, y no fila por fila.
- Se benefician de una alta paralelización (la nube lo realiza de forma automática).
- Se consultan con SQL.
- Algunas permiten particionar las tablas para acelerar consultas específicas.

Ejemplos



Google BigQuery



amazon REDSHIFT

ETLs

Extract, Transform, Load



- Son scripts o tareas que toman datos desde una o más fuentes, los procesan y transforman según sea necesario y, finalmente, los almacenan en otro lugar (usualmente un warehouse).
- Constituyen una parte fundamental del procesamiento de datos en una organización.

ETLs

Modos de ejecución



Periódica

Se programan para ejecutarse cada cierto intervalo (cada 1 hora, 1 día, etc.).



Event-driven

En algunos casos se ejecutan ante eventos, para procesar datos provenientes de un *stream*.

Data Engineering y la nube

Herramientas disponibles

Para implementar ETLs, data warehouses, data lakes, streams de datos, etc., existen herramientas de dos tipos:



Open source



Proveedores cloud (pagadas)

En una organización grande, el trabajo de data engineering suele consistir en combinar ambas para construir pipelines de procesamiento, almacenamiento y uso de los datos.

Ejemplos open source

Herramientas del ecosistema



Plataforma de streaming distribuido de datos.



Procesamiento distribuido de datos (similar a MapReduce).



Orquestación y ejecución de ETLs.



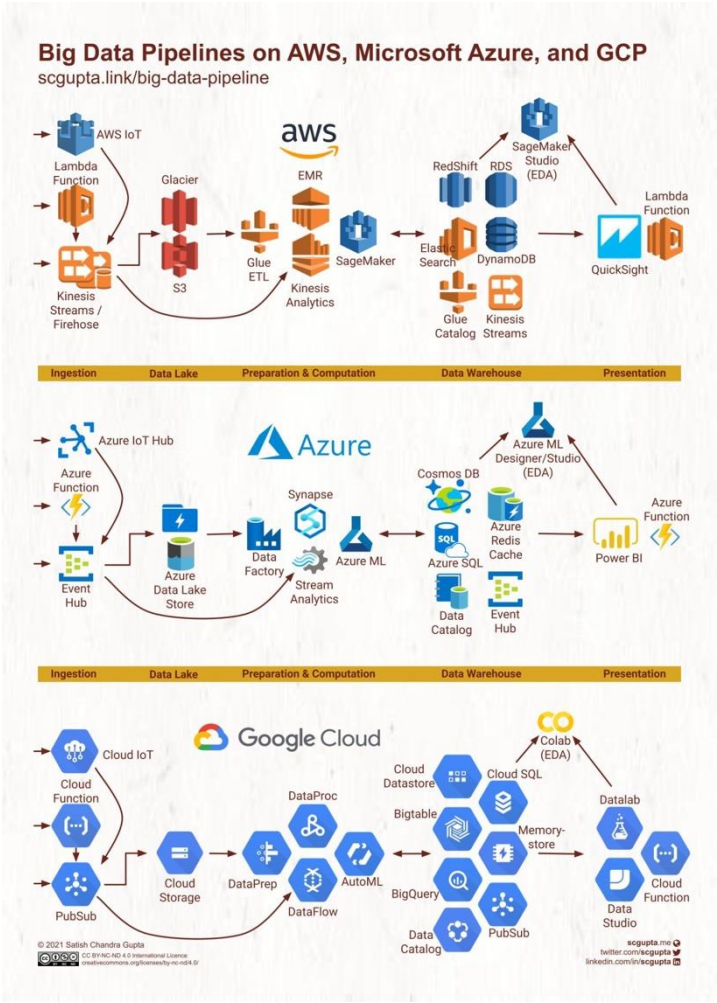
Procesamiento y analítica para datasets de gran tamaño.

Arquitectura completa

Un sistema de datos complejo

En definitiva, un sistema de bases de datos complejo se ve así.

- Pipelines de Big Data en AWS, Microsoft Azure y Google Cloud.
- Cada proveedor combina servicios de ingesta, almacenamiento, procesamiento y visualización.



Ejemplo práctico

Veamos esto en un Jupyter Notebook



Enlaces de interés

Para profundizar

- 14 data pipelines: Amazon Redshift (intermix.io)
- Scalable data analytics pipeline (freeCodeCamp)
- Serverless data analytics pipeline (AWS)
- Data pipeline con Airflow en AWS y GCP (SlideShare)

Fin de la clase

¿Dudas?

